

Лабораторная работа № 3

Дискретно-событийная модель системы массового обслуживания
М/М/1

Куокконен Дарина Андреевна

2026-08-30

Аннотация

Реализована дискретно-событийная модель одноканальной системы массового обслуживания М/М/1. Имитационные оценки загрузки, длины очереди и времён ожидания сопоставлены со стационарными формулами. Проведён нагрузочный эксперимент, показавший нелинейный рост задержки при приближении загрузки к единице. Результаты воспроизводятся в Julia и проверяются тестами.

Содержание

1. Цель и задание
2. Теоретическое введение
3. Среда и организация проекта
4. Алгоритм имитации
5. Базовый эксперимент
6. Нагрузочный эксперимент
7. Производные форматы и Jupyter
8. Проверка и управление версиями
9. Выводы
10. Источники

1. Цель и задание

Цель работы — освоить событийный подход к имитационному моделированию и исследовать систему массового обслуживания М/М/1 с пуассоновским потоком заявок и экспоненциальным временем обслуживания.

В работе требовалось:

1. реализовать календарь событий «приход/уход» без фиксированного шага времени;
2. накопить интегральные статистики длины системы, очереди и занятости прибора;
3. сопоставить оценки с формулами стационарной М/М/1;
4. изучить изменение очереди при росте нагрузки;
5. сформировать CSV, графики, QMD, HTML и выполненные блокноты Jupyter;
6. подтвердить корректность реализации автоматическими тестами.

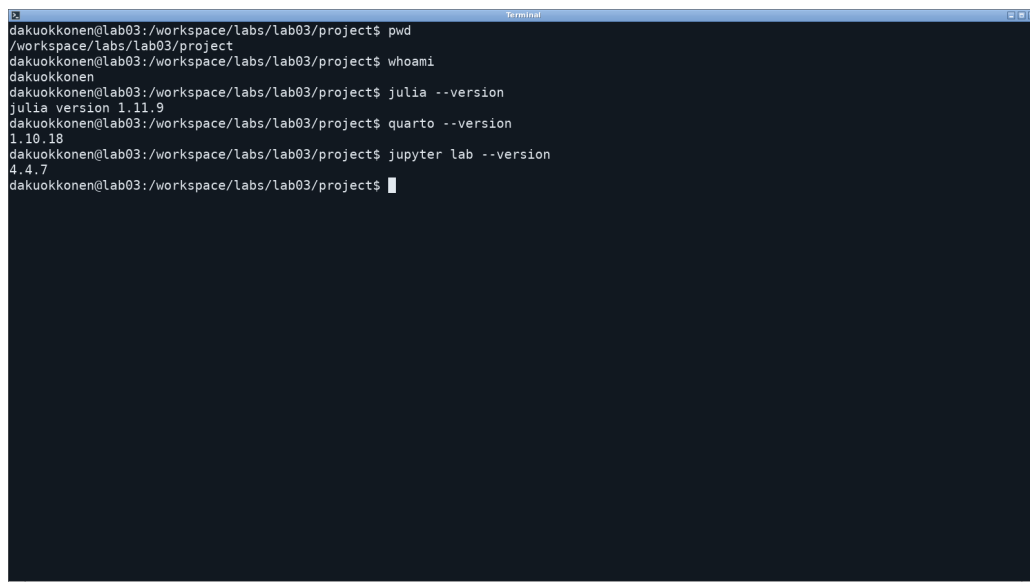
A screenshot of a terminal window with a dark background. The terminal shows a series of commands and their outputs. The user is 'dakuokkonen' and the current directory is '/workspace/labs/lab03/project'. The commands and outputs are: 'pwd' returns '/workspace/labs/lab03/project'; 'whoami' returns 'dakuokkonen'; 'julia --version' returns 'julia version 1.11.9'; 'quarto --version' returns '1.10.18'; 'jupyter lab --version' returns '4.4.7'. The prompt is always 'dakuokkonen@lab03: /workspace/labs/lab03/project\$'.

Рисунок 1: Проверка пользователя, версии Julia и рабочего каталога

2. Теоретическое введение

Обозначим через λ интенсивность поступления заявок, через μ — интенсивность обслуживания, а через

$$\rho = \frac{\lambda}{\mu} \quad (1)$$

— загрузку системы. Стационарный режим существует при $\rho < 1$.

Для М/М/1 среднее число заявок в системе и очереди равно

$$L = \frac{\rho}{1 - \rho}, L_q = \frac{\rho^2}{1 - \rho}. \quad (2)$$

По формуле Литтла времена пребывания и ожидания определяются как

$$W = \frac{1}{\mu - \lambda}, W_q = \frac{\rho}{\mu - \lambda}. \quad (3)$$

Формулы (Уравнение 2) и (Уравнение 3) показывают сингулярный рост показателей при $\rho \rightarrow 1$. Это свойство особенно важно при проектировании информационных и телекоммуникационных систем: небольшое увеличение потока вблизи пропускной способности вызывает резкий рост задержки.

Аналитические характеристики вычисляются отдельной функцией общего модуля. Перед расчётом проверяется условие стационарности, поэтому нестабильная конфигурация не может быть ошибочно использована как контрольный пример.

```
function theoretical_mm1(lambda::Real, mu::Real)
    lambda > 0 || throw(ArgumentError("lambda must be positive"))
    mu > lambda || throw(ArgumentError("stationary M/M/1 requires mu >
lambda"))
    rho = lambda / mu
    return (
        rho=rho,
        mean_system=rho / (1 - rho),
        mean_queue=rho^2 / (1 - rho),
        mean_wait=rho / (mu - lambda),
        mean_sojourn=1 / (mu - lambda),
    )
end
```

Листинг 1 — Аналитические характеристики из `src/queueing_models.jl`

3. Среда и организация проекта

Работа выполнена в Julia 1.11.9. Зависимости закреплены файлами `Project.toml` и `Manifest.toml`. В общем модуле находится математическая и событийная логика, два сценария отвечают за базовый и нагрузочный эксперименты, а тесты обращаются непосредственно к тому же модулю.

Таблица 1: Структура вычислительной части лабораторной работы

Компонент	Назначение
<code>src/queueing_models.jl</code>	аналитические формулы и событийная модель
<code>scripts/01_mm1_queue.jl</code>	базовый эксперимент и сравнение с теорией
<code>scripts/02_load_scan.jl</code>	серия прогонов для семи уровней нагрузки
<code>scripts/tangle.jl</code>	генерация Julia, QMD и IPYNB
<code>test/runtests.jl</code>	проверки формул, ограничений и статистик

```
dakuokkonen@lab03:/workspace/labs/lab03/project$ find src scripts test -maxdepth 2 -type f | sort
scripts/01_mm1_queue.jl
scripts/01_mm1_queue/01_mm1_queue.jl
scripts/02_load_scan.jl
scripts/02_load_scan/02_load_scan.jl
scripts/setup_environment.jl
scripts/tangle.jl
src/project.jl
src/queueing_models.jl
test/runtests.jl
dakuokkonen@lab03:/workspace/labs/lab03/project$ sed -n '1,100p' Project.toml
name = "project"
uuid = "83e4a75a-c85a-4ea1-9c6c-441f1ff60d25"
authors = ["dakuokkonen"]
version = "3.0.0"

[deps]
CSV = "336ed68f-0bac-5ca0-87d4-7b16caf5d00b"
DataFrames = "a93c6f00-e57d-5684-b7b6-d8193f3e46c0"
DrWatson = "634d3b9d-ee7a-5ddf-bec9-22491aa816e1"
IJulia = "7073ff75-c697-5162-941a-fcdaad2a7d2a"
Literate = "98b081ad-f1c9-55d3-8b20-4c87d4299306"
Plots = "91a5bcdd-55d7-5caf-9e0b-520d859cae80"
Quarto = "d7167be5-f61b-4dc9-b75c-ab62374668c5"
Random = "9a3f8284-a2c9-5f02-9a11-845980a1fd5c"
Statistics = "10745b16-79ce-11e8-11f9-7d13ad32a3b2"

[compat]
julia = "1.11"
dakuokkonen@lab03:/workspace/labs/lab03/project$
```

Рисунок 2: Состав Julia-проекта и закреплённые зависимости

3.1 Типы данных и интерфейс модуля

module QueueingModels

using Random
using Statistics

export MM1Result, theoretical_mm1, simulate_mm1

struct MM1Result
 arrival_rate::Float64
 service_rate::Float64
 horizon::Float64
 arrivals::Int
 served::Int
 lost::Int
 utilization::Float64
 mean_system::Float64
 mean_queue::Float64
 mean_wait::Float64
 mean_sojourn::Float64
 times::Vector{Float64}
 queue_lengths::Vector{Int}
 waits::Vector{Float64}
end

Листинг 2 — Объявления модуля и типы данных из src/queueing_models.jl

4. Алгоритм имитации

Модель хранит времена ближайшего поступления и ухода. На каждом шаге выбирается более раннее событие. Между соседними событиями состояние постоянно, поэтому площади под траекториями $N(t)$ и $Q(t)$ накапливаются точно как суммы прямоугольников.

Вектор queue_arrivals содержит времена прихода всех заявок в системе, включая обслуживаемую. Поэтому число ожидающих равно $\max(\text{length}(\text{queue_arrivals}) - 1,$

0). Поле `service_start` хранит начало текущего обслуживания; `next_departure = Inf` означает свободный прибор. В типе результата отдельно сохранены временные средние и выборка ожиданий.

```
function simulate_mm1(lambda::Real, mu::Real; horizon::Real=1000.0,
    capacity::Integer=typemax{Int}, seed::Integer=202603)
    lambda > 0 || throw(ArgumentError("lambda must be positive"))
    mu > 0 || throw(ArgumentError("mu must be positive"))
    horizon > 0 || throw(ArgumentError("horizon must be positive"))
    capacity >= 1 || throw(ArgumentError("capacity must be at least one"))

    rng = MersenneTwister(seed)
    next_arrival = randexp(rng) / lambda
    next_departure = Inf
    queue_arrivals = Float64[]
    service_start = 0.0
    t = 0.0
    last_t = 0.0
    area_system = 0.0
    area_queue = 0.0
    busy_area = 0.0
    arrivals = served = lost = 0
    waits = Float64[]
    sojourns = Float64[]
    times = Float64[0.0]
    queue_lengths = Int[0]

    while true
        event_time = min(next_arrival, next_departure)
        event_time > horizon && break
        n_system = length(queue_arrivals)
        dt = event_time - last_t
        area_system += n_system * dt
        area_queue += max(n_system - 1, 0) * dt
        busy_area += (n_system > 0) * dt
        t = event_time
        last_t = t

        if next_arrival <= next_departure
            arrivals += 1
            if length(queue_arrivals) < capacity
                push!(queue_arrivals, t)
                if length(queue_arrivals) == 1
                    service_start = t
                    next_departure = t + randexp(rng) / mu
                end
            else
                lost += 1
            end
            next_arrival = t + randexp(rng) / lambda
        else
            arrival_time = popfirst!(queue_arrivals)
            push!(waits, service_start - arrival_time)
            push!(sojourns, t - arrival_time)
            served += 1
            if isempty(queue_arrivals)
                next_departure = Inf
            else
                service_start = t
                next_departure = t + randexp(rng) / mu
            end
        end
        push!(times, t)
        push!(queue_lengths, max(length(queue_arrivals) - 1, 0))
    end
end
```

```

end

dt = horizon - last_t
n_system = length(queue_arrivals)
area_system += n_system * dt
area_queue += max(n_system - 1, 0) * dt
busy_area += (n_system > 0) * dt

return MM1Result(
    Float64(lambda), Float64(mu), Float64(horizon), arrivals, served, lost,
    busy_area / horizon, area_system / horizon, area_queue / horizon,
    isempty(waits) ? NaN : mean(waits),
    isempty(sojourns) ? NaN : mean(sojourns),
    times, queue_lengths, waits,
)
end

```

Листинг 3 — Полная функция событийного моделирования из `src/queueing_models.jl`

Интервалы формируются как $\text{randexp}(\text{rng}) / \text{rate}$. Начальные числа генератора фиксируются, что обеспечивает повторяемость. Время ожидания каждой обслуженной заявки вычисляется как разность между началом обслуживания и поступлением.

При приходе счётчик `arrivals` увеличивается даже для отклонённой заявки. Если вместимость не исчерпана, время поступления добавляется в конец FIFO; для первой заявки сразу назначается уход. При уходе удаляется первый элемент, накапливаются ожидание и время в системе, затем назначается следующее обслуживание.

Перед изменением состояния учитывается весь интервал от предыдущего события. После выхода из цикла добавляется остаток до `horizon`: без него средние длины и занятость были бы занижены. Площади делятся на длительность прогона, а ожидания усредняются только по обслуженным заявкам. Последние незавершённые заявки в эту выборку не входят; для короткого прогона это источник смещения. В начальный момент система пуста, отдельный период разогрева здесь не удаляется.

5. Базовый эксперимент

Для сопоставления с учебной постановкой выбраны

$$\lambda=30, \mu=33, T=1000,$$

поэтому $\rho=0,9091$. За один прогон поступило и было обслужено 29 825 заявок; потерь при практически неограниченном буфере не возникло.

Базовый сценарий формирует единую таблицу теоретических и имитационных значений, после чего рассчитывает относительную ошибку каждой характеристики.

```

# # Система массового обслуживания M/M/1
#
# **Цель:** выполнить дискретно-событийное моделирование одноканальной СМО,
# сравнить статистические оценки с формулами теории очередей.

using DrWatson
@quickactivate "project"
ENV["GKSwttype"] = "100"
using CSV, DataFrames, Plots, Statistics
include(scrdir("queueing_models.jl"))
using .QueueingModels

```

```

name = "01_mm1_queue"
mkpath(datadir(name)); mkpath(plotsdir(name))

lambda, mu = 30.0, 33.0
result = simulate_mm1(lambda, mu; horizon=1000.0, capacity=10_000, seed=202603)
theory = theoretical_mm1(lambda, mu)

summary = DataFrame(metric=["rho", "L", "Lq", "W", "Wq"],
    theory=[theory.rho, theory.mean_system, theory.mean_queue,
theory.mean_sojourn, theory.mean_wait],
    simulation=[result.utilization, result.mean_system, result.mean_queue,
result.mean_sojourn, result.mean_wait])
summary.relative_error = abs.((summary.simulation .- summary.theory) ./
summary.theory)
CSV.write(datadir(name, "summary.csv"), summary)
CSV.write(datadir(name, "waits.csv"), DataFrame(wait=result.waits))

println("=== СМО М/М/1: базовый эксперимент ===")
println("lambda = $lambda, mu = $mu, rho = $(round(theory.rho; digits=4))")
println("Заявок: $(result.arrivals), обслужено: $(result.served), потеряно: $
(result.lost)")
show(summary; allrows=true, allcols=true); println()

default(fontfamily="DejaVu Sans", linewidth=2.4, framestyle=:box,
gridalpha=0.22,
    left_margin=5 * Plots.mm)
limit = searchsortedlast(result.times, 20.0)
p1 = plot(result.times[1:limit], result.queue_lengths[1:limit];
seriestype=:steppost,
    xlabel="Время", ylabel="Длина очереди", label="Q(t)",
    title="Фрагмент траектории очереди М/М/1", size=(1000,620))
savefig(p1, plotsdir(name, "queue_trajectory.png"))

p2 = histogram(result.waits; bins=50, normalize=:pdf, alpha=0.65,
    xlabel="Время ожидания", ylabel="Плотность", label="моделирование",
    title="Распределение времени ожидания", size=(1000,620))
savefig(p2, plotsdir(name, "wait_histogram.png"))

p3 = plot(summary.metric, summary.theory; marker=:circle, markersize=8,
    label="теория", ylabel="Значение", title="Теория и имитация М/М/1",
    size=(1000,620))
plot!(p3, summary.metric, summary.simulation; marker=:diamond, markersize=7,
    linestyle=:dash, label="моделирование")
savefig(p3, plotsdir(name, "theory_vs_simulation.png"))

# Полученные оценки близки к стационарным формулам; небольшое отличие связано
# с конечной длительностью одного случайного прогона.

```

Листинг 4 — Полный сценарий базового эксперимента из scripts/01_mm1_queue.jl

Таблица 2: Сопоставление теоретических и имитационных оценок

Показатель	Теория	Имитация	Относительная ошибка
ρ	0,9091	0,9146	0,60%
L	10,0000	9,5247	4,75%
L_q	9,0909	8,6101	5,29%
W	0,3333	0,3194	4,19%
W_q	0,3030	0,2887	4,73%

Расхождение в Таблица 2 объясняется конечной длительностью случайного прогона. Все пять оценок согласуются с теорией с ошибкой не более 5,3%.

Столбцы theory и simulation имеют одинаковый порядок метрик. В relative_error хранится доля, а в таблице отчёта она переведена в проценты. Помимо summary.csv, сценарий сохраняет все наблюждённые ожидания в waits.csv. Траектория изображается ступенчатой линией на первых 20 единицах времени; гистограмма нормирована как плотность. Все три рисунка создаются из того же объекта result, без повторного случайного прогона.

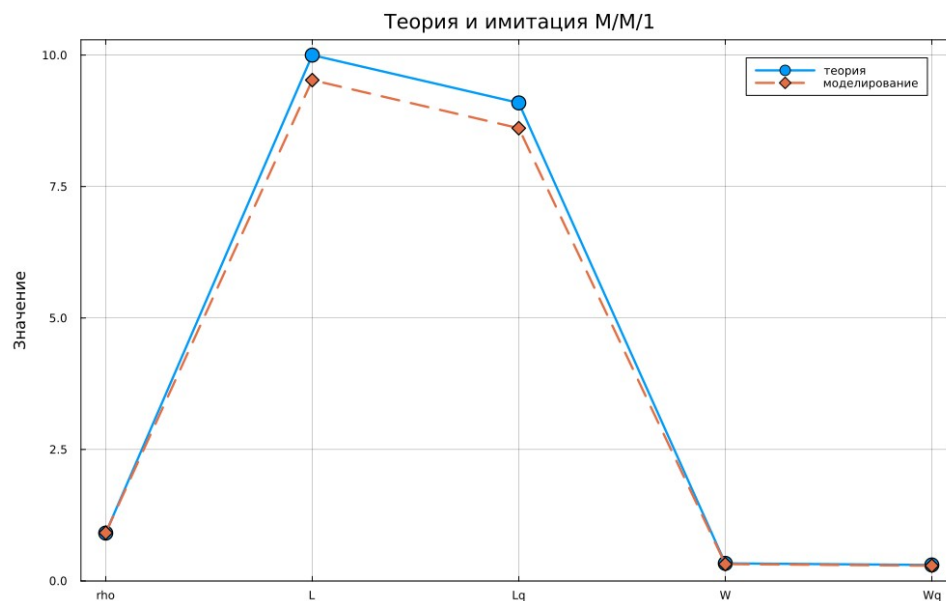


Рисунок 3: Сопоставление теории и имитации M/M/1

Траектория очереди является кусочно-постоянной: приход увеличивает её, а уход уменьшает. На коротком фрагменте видны случайные серии заявок и периоды разгрузки прибора.

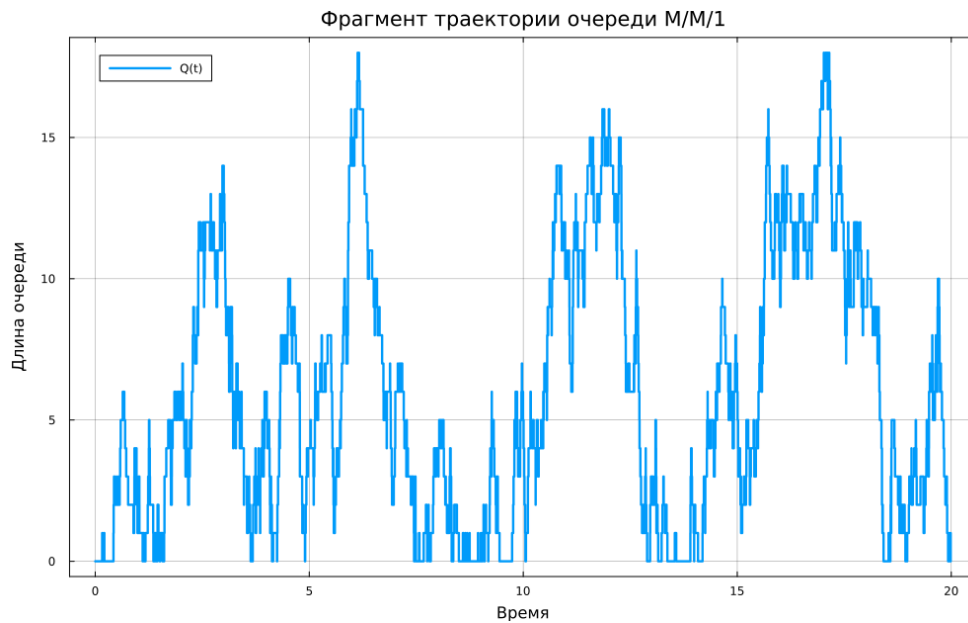


Рисунок 4: Фрагмент траектории длины очереди

Распределение времени ожидания асимметрично: большинство заявок обслуживается с небольшой задержкой, но формируется длинный правый хвост.

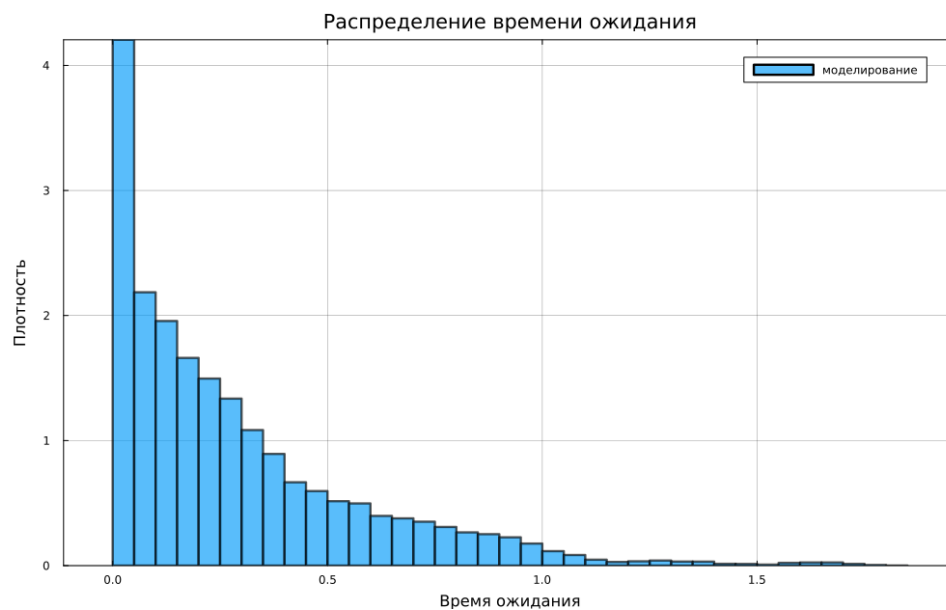


Рисунок 5: Гистограмма времени ожидания

6. Нагрузочный эксперимент

Интенсивность обслуживания зафиксирована на уровне $\mu=33$, а λ изменялась от 10 до 32,5. Для каждой точки выполнено 30 независимых прогонов продолжительностью 1500 единиц модельного времени.

```
# # Нагрузочный эксперимент для М/М/1
```

```
#
```

```
# **Цель:** показать нелинейный рост очереди при приближении загрузки к
```

единице.

```
using DrWatson
@quickactivate "project"
ENV["GKSwttype"] = "100"
using CSV, DataFrames, Plots, Statistics
include(scrdir("queueing_models.jl"))
using .QueueingModels

name = "02_load_scan"
mkpath(datadir(name)); mkpath(plotsdir(name))
mu = 33.0
lambdas = [10.0, 20.0, 25.0, 28.0, 30.0, 31.5, 32.5]
rows = NamedTuple[]

for (i, lambda) in enumerate(lambdas)
    runs = [simulate_mm1(lambda, mu; horizon=1500.0, seed=202603 + 100i + r)
    for r in 1:30]
    theory = theoretical_mm1(lambda, mu)
    push!(rows, (lambda=lambda, rho=theory.rho,
        theory_Lq=theory.mean_queue,
        simulation_Lq=mean(getfield.(runs, :mean_queue)),
        simulation_Wq=mean(getfield.(runs, :mean_wait)),
        utilization=mean(getfield.(runs, :utilization)))
end

scan = DataFrame(rows)
CSV.write(datadir(name, "load_scan.csv"), scan)
println("=== Нагрузочный эксперимент M/M/1 ===")
show(scan; allrows=true, allcols=true); println()

default(fontfamily="DejaVu Sans", linewidth=2.4, framestyle=:box,
gridalpha=0.22,
left_margin=5 * Plots.mm)
p1 = plot(scan.rho, scan.theory_Lq; label="теория", xlabel="Загрузка rho",
ylabel="Средняя длина очереди", title="Рост очереди при rho → 1",
size=(1000, 620), marker=:circle)
plot!(p1, scan.rho, scan.simulation_Lq; label="моделирование", marker=:diamond)
savefig(p1, plotsdir(name, "load_curve.png"))

p2 = plot(scan.rho, scan.simulation_Wq; label="Wq", xlabel="Загрузка rho",
ylabel="Среднее ожидание", title="Задержка в зависимости от нагрузки",
size=(1000, 620), marker=:circle, color=:red)
savefig(p2, plotsdir(name, "wait_curve.png"))

# При rho, близком к единице, статистическая вариативность возрастает вместе
# со средней длиной очереди и временем ожидания.
```

Листинг 5 — Полный сценарий нагрузочного эксперимента из
scripts/02_load_scan.jl

Таблица 3: Результаты нагрузочного эксперимента

λ	ρ	L_q (теория)	L_q (модель)	W_q (модель)
10,0	0,3030	0,1318	0,1314	0,0132
20,0	0,6061	0,9324	0,9199	0,0461
25,0	0,7576	2,3674	2,3502	0,0940
28,0	0,8485	4,7515	4,7150	0,1682
30,0	0,9091	9,0909	9,0916	0,3028
31,5	0,9545	20,0455	20,2187	0,6422
32,5	0,9848	64,0152	65,3523	2,0059

Для каждого уровня нагрузки усредняются 30 значений mean_queue и mean_wait; это не объединение отдельных ожиданий в одну выборку. Индекс уровня и номер повтора включены в seed. Поэтому строка $\lambda=30$ здесь отличается от одиночного базового опыта: $L_q=9,0916$, а не 8,6101. Разные начальные числа позволяют повторить серию без совпадения траекторий. Доверительные интервалы в данном эксперименте не рассчитывались, поэтому таблица характеризует средние оценки, а не точность с заданной вероятностью.

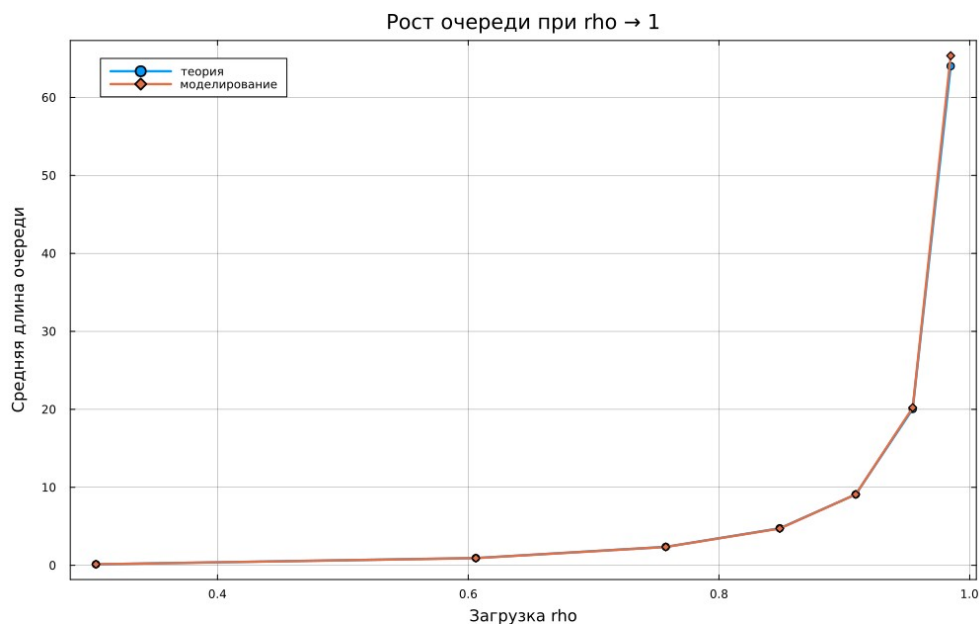


Рисунок 6: Средняя очередь при изменении загрузки

При $\rho=0,9848$ средняя очередь превышает 65 заявок, тогда как при $\rho=0,7576$ она близка к 2,35. Следовательно, резерв пропускной способности нельзя оценивать линейно.

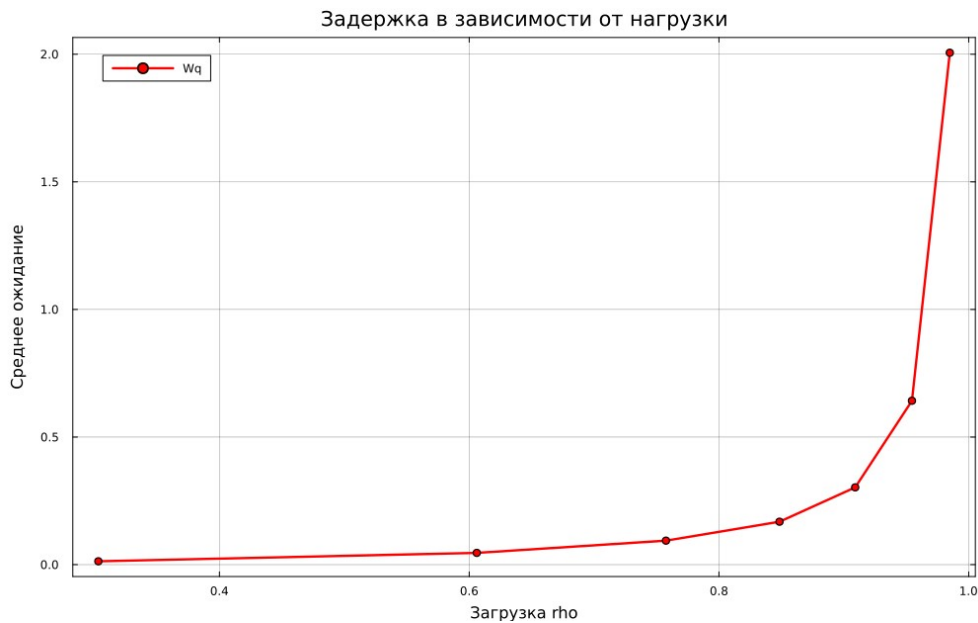


Рисунок 7: Среднее ожидание при изменении загрузки

7. Производные форматы и Jupyter

Сценарий `scripts/tangle.jl` использует `Literate.jl` и для каждого литературного исходника формирует чистый Julia-файл, Quarto-документ и Jupyter-ноутбук. Полученные IPYNB выполняются с ядром Julia лабораторной работы, поэтому в них сохраняются реальные таблицы и текстовый вывод.

```
using DrWatson
@quickactivate "project"
using Literate

function generate_formats(source::AbstractString)
    name = splitext(basename(source))[1]
    foreach(mkpath, (scriptsdir(name), projectdir("markdown", name),
projectdir("notebooks", name)))
        Literate.script(source, scriptsdir(name); name=name, credit=false)
        Literate.notebook(source, projectdir("notebooks", name); name=name,
execute=false, credit=false)
        Literate.markdown(source, projectdir("markdown", name); name=name,
flavor=Literate.QuartoFlavor(), credit=false)
    end
end

foreach(generate_formats, ARGS)
```

Листинг 6 — Генерация производных форматов в `scripts/tangle.jl`

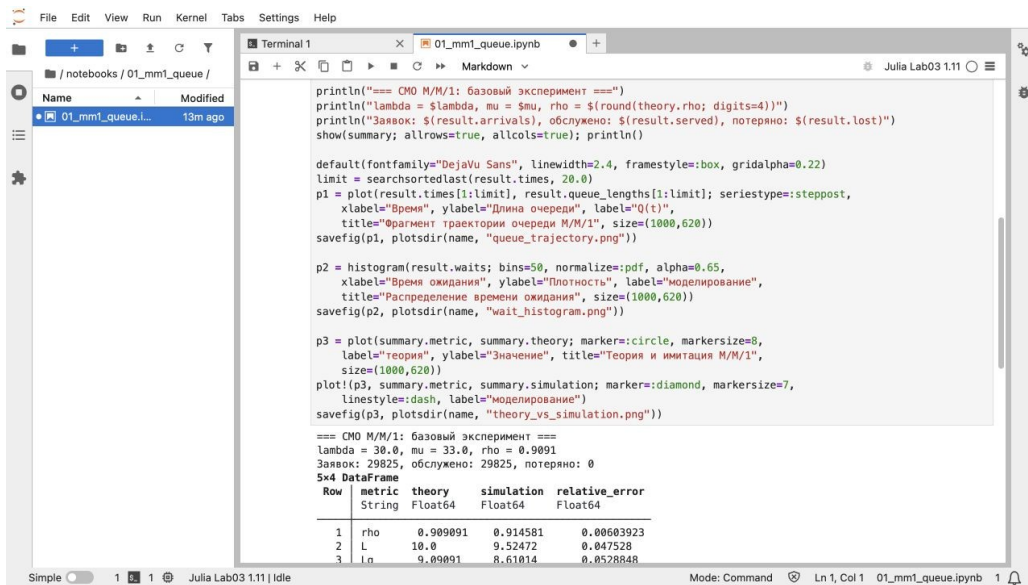


Рисунок 8: Выполненный блокнот базового эксперимента $M/M/1$

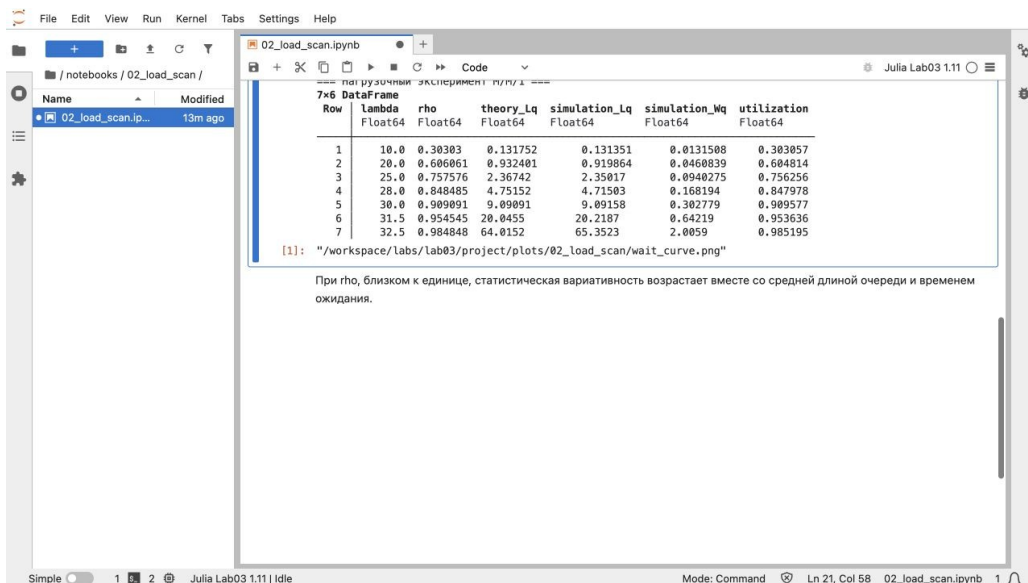


Рисунок 9: Выполненный блокнот нагрузочного эксперимента

8. Проверка и управление версиями

Тесты проверяют известный случай $\lambda=3$, $\mu=5$, неотрицательность очереди и ожиданий, отсутствие потерь при неограниченном буфере, а также близость загрузки и L к аналитическим значениям.

```

using Test
include(joinpath(@__DIR__, "..", "src", "queueing_models.jl"))
using .QueueingModels

@testset "M/M/1 formulas" begin
    t = theoretical_mm1(3.0, 5.0)
end

```

```

@test t.rho == 0.6
@test isapprox(t.mean_system, 1.5)
@test isapprox(t.mean_queue, 0.9)
@test isapprox(t.mean_sojourn, 0.5)
@test_throws ArgumentError theoretical_mm1(5.0, 5.0)
end

@testset "M/M/1 simulation" begin
    r = simulate_mm1(3.0, 5.0; horizon=20_000.0, seed=42)
    @test r.arrivals >= r.served
    @test r.lost == 0
    @test abs(r.utilization - 0.6) < 0.03
    @test abs(r.mean_system - 1.5) < 0.15
    @test all(>=(0), r.queue_lengths)
    @test all(>=(0), r.waits)
end

```

Листинг 7 — Статистические проверки в test/runtests.jl

Все 11 проверок пройдены: пять проверок стационарных формул и шесть проверок имитации. Для тестового прогона используются $\lambda=3$, $\mu=5$, $T=20000$ и seed 42, то есть параметры отличаются от демонстрационного опыта. Допуски 0,03 для загрузки и 0,15 для среднего числа заявок учитывают случайность. Эти тесты не являются доказательством стационарности любого набора параметров.

```

dakuokkonen@lab03:/workspace/labs/lab03/project$ make test
julia --project=. test/runtests.jl
Test Summary: | Pass Total Time
M/M/1 formulas | 5 5 0.0s
Test Summary: | Pass Total Time
M/M/1 simulation | 6 6 0.5s
dakuokkonen@lab03:/workspace/labs/lab03/project$

```

Рисунок 10: Результат выполнения автоматических проверок

8.1 Воспроизведение расчётов и сборка материалов

Команды выполняются в терминале контейнера из каталога лабораторной.

```

cd /workspace/labs/lab03/project
julia --project=. -e 'using Pkg; Pkg.instantiate()'
make setup
make all
make test
jupyter lab --ip=0.0.0.0 --port=8888 --no-browser

```

Листинг 8 — Восстановление окружения, расчёты и запуск JupyterLab

Цель `tangle` создаёт производные исходники, `run` выполняет оба эксперимента, `notebooks` исполняет IPYNB, `docs` собирает вычислительные HTML-страницы, а `test` запускает проверки общего модуля. В JupyterLab открываются файлы из `notebooks`; ядро Julia устанавливается командой `make setup`.

```
cd /workspace/labs/lab03/report
make all
cd ../presentation
make all
```

Листинг 9 — Сборка отчёта DOCX/PDF и HTML-презентации

Quarto создаёт DOCX и HTML, LibreOffice — PDF. PPTX редактируется отдельно; цель `make all` презентации пересобирает HTML и проверяет наличие PPTX.

9. Выводы

Дискретно-событийная модель М/М/1 реализована и проверена. Для базового режима $\rho = 0,9091$ имитационные характеристики согласуются со стационарными формулами с максимальной относительной ошибкой 5,3%. Нагрузочный эксперимент подтвердил резкий рост очереди и ожидания при приближении λ к μ . Получен полностью воспроизводимый Julia-проект с таблицами, графиками, документацией, блокнотами и тестами.

10. Источники

1. Gross D., Shortle J. F., Thompson J. M., Harris C. M. *Fundamentals of Queueing Theory*. Wiley, 2018.
2. Julia Documentation. <https://docs.julialang.org/>
3. Literate.jl Documentation. <https://fredrikekre.github.io/Literate.jl/>
4. Quarto Documentation. <https://quarto.org/docs/>